

Grounded

A RAG-driven technical screening system. It reads a candidate resume, decides what is worth testing for a target role, retrieves the supporting passages from that role's textbook corpus, and asks questions that are grounded in what it retrieved — then grades the answers against the same passages.

Assignment	PG-AGI — AI/ML & Backend Engineering Intern
Deliverable	System design, architecture and key decisions
Stack	FastAPI · SQLAlchemy · NumPy · Next.js 15 · Gemini
Repository	github.com/hrithiknl17/ai-technical-screening

1. Contents

2	Problem framing and the one design commitment
3	System architecture
4	The pipeline, stage by stage
5	RAG design decisions
6	Backend and API design
7	Data model and persistence
8	Frontend design
9	Failure modes and degradation
10	Testing strategy
11	Deployment
12	Trade-offs, limits and what comes next

2. Problem framing and the one design commitment

The brief asks for a screening system whose questions are *not predefined* — they must fall out of the resume, the role and a role-specific corpus. That framing makes one property the whole system has to defend: **every question must be traceable to retrieved text**. A system that quietly falls back to a stored question bank, or to whatever the model happens to know about gradient boosting, satisfies the demo but not the brief.

So the commitment is: **no question exists without a retrieval trace**. Each stored question carries the query that produced it, the chunk ids returned, their hybrid/dense/lexical scores, their source book and page range, and an excerpt. The UI surfaces this under "*Why this question*". Grading then reads the *same* passages, so the grader and the question share one ground truth rather than two independent guesses.

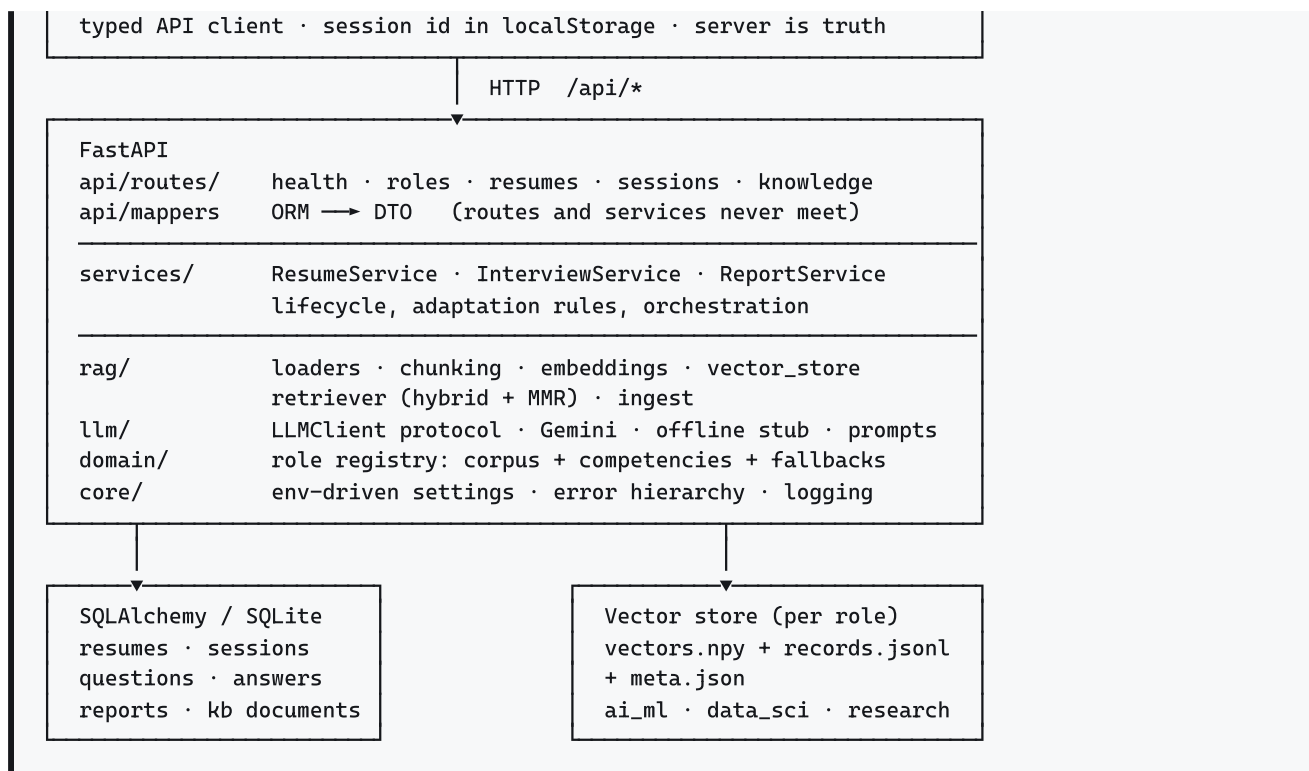
Three consequences follow, and they shape most of the decisions in this document:

- A role whose corpus is not indexed **cannot be interviewed**. The API refuses rather than degrading into ungrounded generation.
- Retrieval quality is a first-class concern, not a detail — a vague passage produces a vague question, and no amount of prompt engineering downstream repairs it.
- Numbers in the final report are **computed from stored grades**, never produced by the model. The model is asked to interpret evidence, not to invent it.

3. System architecture

Three tiers, one process boundary. The frontend is a Next.js App Router client; the backend is a FastAPI service with a service layer that owns all business logic; persistence is SQLAlchemy over SQLite (Postgres by changing one environment variable) plus a per-role vector index on disk.

Next.js (App Router) setup → interview → report

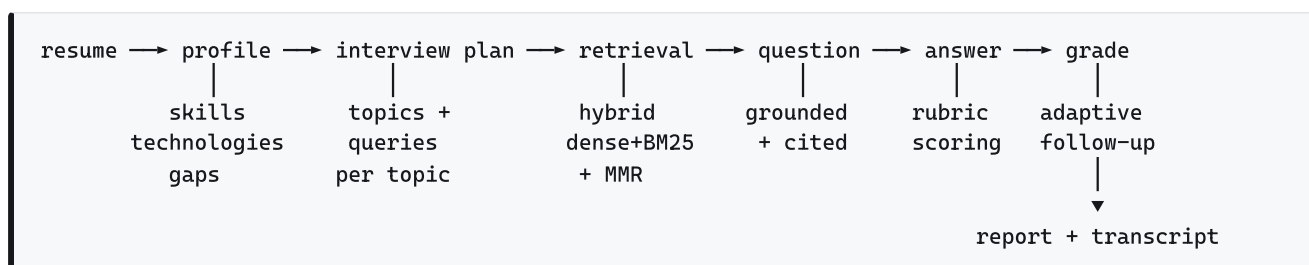


WHY THE LAYERS ARE CUT HERE

Routes translate HTTP into a method call and nothing else. `InterviewService` owns the interview lifecycle — planning, asking, grading, adapting, completing. That split is what makes the adaptive rules testable without a web server, and it is why the offline suite drives the entire flow through the API in under two seconds. `api/mappers.py` converts ORM rows to DTOs in one place, so services never import response schemas and routes never touch the ORM.

Configuration is entirely environment-driven (`app/core/config.py` , a Pydantic `BaseSettings`): model names, embedding provider and dimension, chunk size and overlap, `RETRIEVAL_TOP_K` , `DENSE_WEIGHT` , `RETRIEVAL_MMR_LAMBDA` , question counts, CORS origins, `DATABASE_URL` , `OFFLINE_MODE` . The same image runs in every environment without a code edit.

4. The pipeline, stage by stage



4.1 Candidate entry

The UI lists roles and, per role, the books backing it and the number of indexed chunks. A role with an empty index is not selectable — the refusal is enforced server-side too.

4.2 Resume processing

`POST /api/resumes` accepts PDF, TXT or MD (5 MB cap). Two extraction layers run and are merged:

- **Deterministic pass** — skill dictionary with word-boundary matching (so "R" does not match "React"), date-range union for years of experience, name heuristics. Always available, always right about what it finds.
- **LLM pass** — a structured profile: skills, technologies, domains, seniority, strength signals and **gap signals**.

Merge rule: the model's structure wins, and deterministic values fill anything it left empty. If the LLM is unavailable the deterministic profile stands alone and the interview still runs.

4.3 Context construction — where the resume actually bites

`InterviewService._build_plan` asks the model for N topics given the profile, the role's competencies, and *the section headings actually present in the index*. Each planned topic carries:

- **why it was chosen**, tied to something concrete in the resume;
- **a difficulty** calibrated to seniority;
- **a question type** (conceptual, applied, diagnostic);
- **a retrieval query written in textbook vocabulary** — not interview phrasing.

That last point is the subtle one. "Tell me how you would handle overfitting in your churn model" is a bad retrieval query; "regularisation, bias–variance decomposition, model complexity penalty" is a good one. Query construction and question phrasing are deliberately different jobs, and the plan stores the query so the mapping stays auditable.

4.4 Knowledge retrieval

Hybrid dense + BM25 over the role's collection, MMR re-ranking for diversity, and chunk-level exclusion of everything already used earlier in the same session — so question 5 cannot be a paraphrase of question 2. Details in §5.

4.5 Question generation

Retrieved passages + candidate profile + the last few turns go into the question prompt. The model returns the question, its rationale, the 3–5 points a strong answer must contain, and the citations it used. All of it is persisted with the question row.

4.6 Interactive interview and adaptation

Answers are graded synchronously against the expected points *and* the retrieved passages. After each answer two adaptation rules fire:

- the difficulty of the next planned topic moves one step up or down;
- the grader may request a follow-up — budgeted at two per interview — pushing deeper after a strong answer, or asking for specifics after a vague one.

The budget matters: without it a weak candidate spirals into an interview made entirely of follow-ups on one topic, and coverage collapses.

4.7 Final output

The report merges statistics computed from stored grades — means, per-topic scores, difficulty mix, follow-up count, which books were drawn on — with an LLM narrative. The transcript keeps every question's provenance attached.

5. RAG design decisions

5.1 Chunking: structure-aware packing, 1400 chars with 220 overlap

Paragraph boundaries first; sentence packing only when a paragraph is oversized; overlap started at a sentence boundary rather than mid-word. 1400 characters is roughly one full textbook argument — a definition plus the paragraph that motivates it — which is the unit a good question is built from. Much smaller and the retrieved context is a fragment that cannot support a question; much larger and the embedding averages several ideas and retrieval goes vague. Chunks under 220 characters are folded into their predecessor instead of being indexed as noise. Every chunk keeps its page range, which is what makes citation possible at all.

5.2 A quality filter before embedding

Textbook PDFs extract tables of contents, bibliographies, index pages and equation soup. Those embed perfectly well and then surface as "context", producing nonsense questions. `is_useful_chunk` rejects short blocks, blocks with a low alphabetic ratio, dot-leader TOC lines and reference-list patterns. On this corpus it drops roughly 8% of chunks — cheaper embeddings *and* better retrieval.

5.3 Embeddings: one interface, three providers

PROVIDER	WHAT IT IS	INDEX 3,945 CHUNKS	WHY PICK IT
<code>local</code> (default)	Model2Vec static model, <code>potion-base-8M</code> , 256-d, CPU only	~40 s	reproducible, no key, no quota
<code>gemini</code>	<code>gemini-embedding-001</code> at 768-d, task-typed	hours on free tier	best retrieval quality
<code>hashing</code>	hashed n-grams, zero dependencies	seconds	tests, CI, air-gapped runs

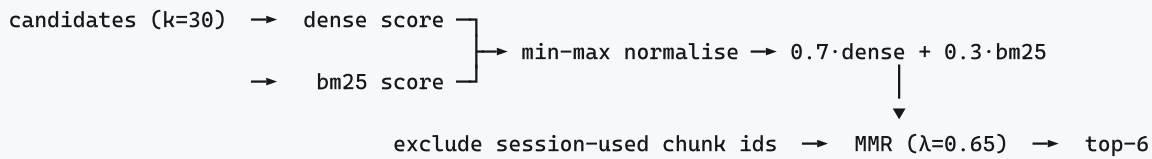
The hosted encoder is the better one — documents embed with `RETRIEVAL_DOCUMENT`, queries with `RETRIEVAL_QUERY`, and that asymmetry measurably helps. But its free tier shares a *global* per-minute embed quota, and a book-sized corpus takes hours behind it. That is a poor default for something a reviewer must be able to reproduce, so the local static model is the default and the hosted one is a single environment variable away. Static embeddings lose ground on subtle paraphrase — which is part of why retrieval is hybrid rather than dense-only.

A collection records the dimension it was built with, and the store rejects a query vector that does not match. Switching provider tells you to re-ingest instead of silently returning nonsense.

5.4 Retrieval: hybrid dense + BM25, then MMR

Dense embeddings match paraphrase ("regularisation" $\approx$ "penalising complexity"). BM25 matches the exact token a resume brags about ("XGBoost", "EM algorithm") — a token a 256-d static embedding will happily smear into a neighbourhood of vaguely-similar concepts. Both scores are min–max normalised per query and blended (`DENSE_WEIGHT`, default 0.7 dense / 0.3 lexical).

MMR ($\lambda = 0.65$) then trades a little relevance for coverage. Books repeat themselves; a pure top-k returns five paraphrases of one paragraph, which yields five near-identical questions. Chunks already used in the session are excluded outright before scoring.



5.5 Vector store: a persistent flat index behind an interface

The corpora are books: 10^3 – 10^4 chunks. An exact normalised matrix–vector product over $10k \times 768$ floats is about 7 MFLOPs — well under a millisecond in NumPy — so an ANN index would add a dependency and an approximation for no measurable gain at this scale. The partitioning that *does* matter is per role, and it is explicit: one collection per role, so a question for one role can never retrieve another role's corpus.

`VectorStore` is a Protocol. Swapping in Chroma, Qdrant or pgvector is a one-class change and the retriever would not notice. That is the honest form of "we could scale this" — an interface that has exactly one implementation today, rather than a distributed system nobody needs.

INDEXED CORPORA

ROLE	CORPUS	CHUNKS
AI/ML Engineer	Mitchell, <i>Machine Learning</i> ; <i>ML for Absolute Beginners</i>	1,047
Data Scientist / Applied ML	<i>Introduction to ML with Python</i> ; Brownlee, <i>Master ML Algorithms</i>	899
ML Researcher (Theoretical)	Bishop, <i>Pattern Recognition and ML</i> ; <i>AI, ML & Deep Learning</i>	1,999

5.6 Prompts in one module

`app/llm/prompts.py` holds every prompt and its response schema. Gemini's `responseSchema` constrains the output shape, which removes the usual "parse JSON out of prose" failure mode; `parse_json` remains tolerant of fenced blocks as a backstop. Keeping prompts in one file means the contract between the service layer and the model is reviewable in one place instead of scattered through business logic.

6. Backend and API design

METHOD	PATH	PURPOSE
GET	<code>/api/health</code>	liveness plus dependency status (DB, LLM, per-role index size)
GET	<code>/api/roles</code>	roles with corpus, competencies, index readiness
GET	<code>/api/roles/{slug}/knowledge</code>	index metadata for one role
POST	<code>/api/resumes</code>	multipart upload → parsed profile
POST	<code>/api/resumes/text</code>	same, from pasted text

METHOD	PATH	PURPOSE
POST	/api/sessions	start an interview (builds the plan, asks Q1)
GET	/api/sessions/{id}	full session state incl. transcript and traces
POST	/api/sessions/{id}/next-question	pending question, or generate the next (204 when exhausted)
POST	/api/sessions/{id}/answers	submit an answer → grade + updated session
POST	/api/sessions/{id}/complete	close the interview, build the report
GET	/api/sessions/{id}/report	fetch the report
POST	/api/knowledge/search	run a retrieval query directly (introspection)

LIFECYCLE, NOT CRUD

The session endpoints are named for interview stages rather than for rows. `next-question` is idempotent — it returns the pending question if one exists and only generates when none does — so a refresh or a double-click cannot skip a question or burn LLM quota. Answering a question that already has an answer is a 409 rather than a silent overwrite.

ONE ERROR CONTRACT

Every failure returns `{"code", "message", "details"}`, produced by a single exception handler translating a domain error hierarchy that services raise without knowing HTTP exists:

DOMAIN ERROR	HTTP	TYPICAL CAUSE
<code>ValidationError</code>	422	resume too short, question count out of range
<code>NotFoundError</code>	404	unknown role slug or session id
<code>ConflictError</code>	409	answering an already-answered question
<code>UpstreamError</code>	502	the LLM failed past its retries
<code>KnowledgeBaseError</code>	503	the role's index is missing or empty

7. Data model and persistence

SQLAlchemy 2.0 typed models over SQLite; `DATABASE_URL` is the only thing standing between this and Postgres. The schema mirrors the pipeline rather than the UI.

<code>resumes</code>	<code>id · filename · raw_text · profile(JSON)</code>
<code> 1-n</code>	
<code>interview_sessions</code>	<code>id · resume_id · role_slug · status · planned_questions</code>
<code> 1-n</code>	<code>plan(JSON: topics, queries, difficulty, status) · completed_at</code>
<code>questions</code>	<code>id · session_id · position · text · topic · difficulty</code>
<code> 1-1</code>	<code>question_type · origin(planned followup) · rationale</code>
<code>answers</code>	<code>id · question_id(unique) · text · time_taken_seconds</code>
	<code>evaluation(JSON: score, covered, missed, follow_up_request)</code>

```
reports          id · session_id(unique) · overall_score · recommendation
                  summary · payload(JSON: per-topic stats, difficulty mix, sources)

knowledge_documents role_slug · filename · content_hash · pages · chunks
```

Two choices are worth calling out.

The retrieval trace is a column, not a log line. Query, chunk ids, scores, source book, page range and excerpt live on the question row. Traceability survives a restart, a log rotation and a redeploy, and the UI reads it from the same place an auditor would.

Structured payloads are JSON columns. The profile, the plan, the evaluation and the report payload are shapes that the LLM contract owns and that will change as prompts change. Normalising them into tables would buy queryability nobody needs here and cost a migration every time a prompt gains a field. The relational spine — the parts that are joined, filtered and constrained — is normalised; the leaves are documents.

`answers.question_id` is `unique`, which makes double-submission a database-level impossibility rather than a race the service layer has to win.

8. Frontend design

Next.js 15 App Router. One stage machine — `setup` → `interview` → `report` — with three stage components (`SetupStage`, `InterviewStage`, `ReportStage`), a `RetrievalTrace` panel and a `Stepper`. The API client is typed against the same DTO shapes the backend returns.

The backend is the source of truth for session state. The client stores only the session id in `localStorage`; a mid-interview refresh re-fetches and returns to the current question. There is no client-side copy of the transcript to drift out of sync, and no way for a reload to lose an answer that was accepted.

The visual language follows a documented token set (white canvas, dark ink, signature colour cards, near-black pill CTA). *"Why this question"* is deliberately an expandable panel on the question itself rather than a debug page — the grounding is a product feature, not developer tooling.

9. Failure modes and degradation

The system pairs a deterministic component with an LLM component at every stage where it can, so losing the model degrades output quality instead of breaking the flow.

FAILURE	BEHAVIOUR
No API key / <code>OFFLINE_MODE=1</code>	hashing embedder + stub LLM returning schema-shaped payloads built from retrieved passages; full flow, storage and UI stay exercisable (this is how the tests run)
LLM rate-limited past retries	answer is stored ungraded ; the interview continues; the report falls back to statistics with a note
LLM profile extraction fails	deterministic dictionary profile stands alone

FAILURE	BEHAVIOUR
Role index missing or empty	503 <code>KnowledgeBaseError</code> ; role is not selectable in the UI — the system refuses to interview from an empty corpus
Embedding dimension mismatch	store rejects the query and tells you to re-ingest, rather than returning nonsense

RATE LIMITING, CONCRETELY

Free-tier limits are the practical constraint on this project, and two mechanisms absorb them. Every client honours a server-supplied `retryDelay` in preference to its own backoff, and the LLM client spaces its own calls (`LLM_REQUESTS_PER_MINUTE` , default 14). The hosted embedder paces itself with AIMD — the inter-batch pause shrinks 10% after each success and grows 60% after a 429 — so a long ingest survives unattended instead of dying on the first burst.

`LLM_MODEL` defaults to `gemini-3.1-flash-lite` , not because it is the strongest model but because free-tier *daily* caps differ enormously per model: `gemini-2.5-flash` allows about 20 generate-content requests per day, which a single interview exhausts. Point `LLM_MODEL` at a stronger model or a paid key for sharper questions.

10. Testing strategy

22 tests, fully offline, no API key needed — `python -m pytest -q` in `backend/` . The suite runs against the hashing embedder and the stub LLM, which is why it can drive the whole API in roughly 1.7 seconds and why it is deterministic enough to assert on.

- **RAG unit tests** — PDF text repair, chunk sizing and overlap, the quality filter, embedder determinism, BM25 ranking, vector-store round-trip and citation format, hybrid retrieval grounding and no-repeat behaviour.
- **Orchestration tests** — plan statuses advance and persist, the interview exhausts correctly, a strong answer earns a budgeted follow-up and pushes the difficulty ladder up, completion drops an unanswered question.
- **API lifecycle and error contracts** — full flow end to end, 409 on double answer, 404 on unknown role or session, 422 on a too-short resume.

The thing being protected is behaviour, not a coverage percentage: the adaptive rules and the grounding guarantees are the parts that silently rot, so those are the parts under test.

11. Deployment

The production build is **single-origin**. `next build` with `STATIC_EXPORT=1` emits a static site into `frontend/out` , and the API mounts it at `/` — after the routers, so `/api/*` always wins. One process, one port, no CORS configuration in production, and the client calls a relative `/api` .

The `Dockerfile` does both stages and bakes the embedding model into the image, so the first request is not a model download. `render.yaml` is a Render Blueprint for the same image: Docker runtime, health check on `/api/health` , `GEMINI_API_KEY` from the dashboard. Any Docker host works identically.

The vector index (~9 MB) **is committed**, so a clone runs immediately without downloading 90 MB of textbooks. The PDFs themselves are not committed; `scripts.fetch_books` then `scripts.ingest_kb --all` rebuilds the index from source. Ingestion is idempotent — a role's collection is rebuilt from whatever is in its folder.

12. Trade-offs, limits and what comes next

ACCEPTED TRADE-OFFS

- **Static local embeddings by default.** Reproducibility beat retrieval quality, because a reviewer has to be able to run this. Hybrid retrieval exists partly to cover the resulting weakness on paraphrase.
- **Flat exact index.** Correct at this scale, wrong at 10^6 chunks. The Protocol is the escape hatch, deliberately unexercised.
- **Synchronous grading.** Simple and observable; it also means a slow model is a slow UI. A queue is the right answer at real volume and the wrong answer for a 48-hour build.
- **SQLite.** One writer, one file, zero setup. `DATABASE_URL` is the whole migration path.
- **CLI-only ingestion.** Adding a corpus is an operator action, not a product feature — honest for now, and the first thing a real deployment would need to fix.

KNOWN LIMITS

- Question quality is bounded by corpus quality; a book that covers a topic badly produces a shallow question about it, and the system has no way to know.
- The grader and the question generator share a model family, so they share its blind spots.
- No authentication — every session is anonymous and reachable by id.
- The host for Burkov's *Hundred-Page ML Book* serves an invalid TLS certificate, so that download usually fails; the AI/ML role is backed by the other two books.

NEXT

- Move ingestion to a background worker with a progress endpoint, so a corpus can be added from the UI rather than the CLI.
- Cache question generation per (topic, chunk-set) to make demos cheap and repeatable.
- A second grader pass for disagreement detection — one model grading, one auditing.
- Voice answers: the answer path is already a plain string, so only the input surface changes.